

Medical Insurance Cost Prediction and Risk Classification Using Machine Learning

Nelson Lee, Mohsin Raja, Benjamin Morgan, Lu Liang, Xiang Mao
ECS 171 – Machine Learning
University of California, Davis
Team 20

Abstract—This paper tackles two related problems in medical insurance: predicting how much a patient will cost and figuring out whether they are high-risk. We use a Kaggle dataset with 100,000 patients and 54 features that cover demographics, health metrics, insurance plan details, and medical history. We tried a range of models on both tasks. We used Ridge and Linear Regression for cost prediction, and Logistic Regression (L1/L2), Random Forest, and Gradient Boosting for classification. Ridge Regression ended up explaining about 96.6% of the variance in annual costs ($R^2 = 0.966$). For binary high-risk classification, Gradient Boosting got a perfect score, literally zero misclassifications on 20,000 test samples. We also ran experiments with a derived three-class insurance quality label and found something interesting: when we left in two features that were basically proxies for the target, accuracy was 99.98%, but removing them dropped it to 43.5%. This was a good lesson in watching out for data leakage.

Index Terms—medical insurance, cost prediction, risk classification, machine learning, gradient boosting, random forest, logistic regression

I. INTRODUCTION AND BACKGROUND

Healthcare spending in the US hit \$4.5 trillion in 2022 and it keeps going up [1]. Insurance companies need to figure out how much each patient is likely to cost so they can set premiums. Hospitals want to know which patients are high-risk so they can step in early with things like chronic disease programs or preventive screenings. And policymakers rely on cost models when designing subsidies and coverage rules.

The way this has traditionally been done is with a handful of variables (age, sex, whether someone smokes, their BMI) plugged into a generalized linear model [6]. That approach is simple and easy to interpret, but it misses a lot. A patient's actual risk depends on a whole combination of things: how many chronic conditions they have, what medications they take, their procedure history, what kind of insurance plan they are on, their income, and so on. There are interactions between these variables that a simple linear model just cannot pick up.

This is where machine learning comes in. Methods like Random Forests [9] and Gradient Boosting [8] are good at handling messy tabular data with mixed feature types. They find interactions automatically and tend to be robust to outliers. Random Forest works by training hundreds of decision trees on random subsets of the data and random subsets of features, then averaging their predictions. This reduces variance and makes the model less likely to overfit compared to a single tree. Gradient Boosting takes a different approach: it builds trees sequentially, where each new tree tries to fix the mistakes

of the previous ones. This iterative correction tends to produce better accuracy but can overfit if you are not careful with the learning rate and tree depth. On the linear side, regularized models like Ridge and Lasso keep things more interpretable while still controlling overfitting. Ridge adds an L2 penalty that shrinks all coefficients toward zero but keeps them all in the model. Lasso uses an L1 penalty that can push some coefficients to exactly zero, which doubles as automatic feature selection. This is useful when you have 50+ features and want to know which ones actually matter. ElasticNet combines both penalties. Logistic regression with L1/L2 penalties extends these same ideas to classification.

Our project has two main goals:

- 1) **Regression**: predict a patient's annual medical cost using their demographic, clinical, insurance, and utilization data.
- 2) **Classification**: classify patients as high-risk vs. low-risk (binary), and also into low/medium/high insurance quality tiers (three-class).

We compared several model families across both tasks, using proper train/validation/test splits and cross-validated hyperparameter tuning. One thing we paid particular attention to was data leakage. We set up three separate classification experiments: one where we left in all features, one where we removed columns that were basically just encoding the target, and one with the original binary risk label. The difference in results was pretty dramatic and taught us a lot about being careful with feature selection.

We also generated learning curves to understand how each model behaves as it sees more training data, whether it is underfitting, overfitting, or somewhere in between. Everything is implemented in Python with scikit-learn [7], and all the code, config files, and results are in a public GitHub repo (link in Section VII). We also built an interactive web dashboard using Next.js that lets you browse model performance, confusion matrices, learning curves, and EDA plots without having to dig through CSV files. We found it useful during development for quickly comparing experiments.

The rest of the paper goes through related work (Section II), the dataset and our EDA findings (Section III), the methodology (Section IV), results (Section V), and conclusions (Section VI).

II. LITERATURE REVIEW

We looked at a few papers on insurance cost prediction to see what had been done before.

Thamarai and Malarvizhi [2] used linear regression, random forest, and gradient boosting on a small dataset of about 1,300 records with 7 features. They found that smoking and BMI mattered the most, and gradient boosting gave the best predictions. The dataset is pretty small though, so it is hard to say how well those results would hold up on larger data.

Boodhun and Jayabalan [3] looked at risk prediction in life insurance with logistic regression and decision trees. Their main takeaway was that feature engineering and regularization matter a lot when you have many clinical features, something we kept in mind when building our pipeline.

Razia and Rao [4] tried SVMs and neural networks for cost estimation. They concluded that ensemble methods tend to beat single models on tabular health data, which lines up with what we saw in our own experiments.

Huang and Chen [5] used XGBoost on a 50,000-patient claims dataset and added SHAP values for interpretability. They showed that you can get high accuracy and still explain what the model is doing, which is important in a healthcare context.

What makes our project different is mainly scale and scope. We have 100,000 patients instead of a few thousand, 54 features instead of a handful, and we do both regression and classification on the same data. We also specifically tested what happens when you remove features that leak information about the target, which none of these papers really addressed.

One common thread across all these papers is that tree-based ensembles (gradient boosting in particular) tend to come out on top for tabular medical data. That motivated us to include both Random Forest and Gradient Boosting alongside the linear models, so we could see for ourselves whether the pattern holds on a much larger dataset. We also wanted to see how big the gap between linear and ensemble methods actually is, since logistic regression is still widely used in clinical settings precisely because it is interpretable.

III. DATASET DESCRIPTION AND EXPLORATORY DATA ANALYSIS

A. Dataset Overview

Our dataset comes from Kaggle and has 100,000 rows and 54 columns. Table I shows how the features break down.

There are no duplicate rows. Only one column has missing values: `alcohol_freq` at about 30%. Everything else is complete, which made preprocessing a lot simpler.

The dataset comes with a built-in binary label called `is_high_risk` that flags 36.8% of patients as high-risk and 63.2% as low-risk. There is also a numeric `risk_score` column that ranges from 0 to 1 and seems to be the continuous version of the binary flag.

For regression, the target is annual medical cost, which ranges from near zero up to about \$60,000. The distribution is right-skewed with a median around \$3,500. For classification,

TABLE I
FEATURE GROUPS IN THE MEDICAL INSURANCE DATASET.

Group	Count	Examples
Demographic	7	age, sex, region, education, marital status, urban/rural
Health / Clinical	10	BMI, blood pressure, HbA1c, LDL, chronic conditions
Insurance Plan	6	plan type, network tier, deductible, co-pay, policy term
Medical Usage	8	visits, hospitalizations, days hospitalized, procedure counts
Financial	6	income, annual medical cost, annual/monthly premium, claims
Binary Flags	12	hypertension, diabetes, asthma, COPD, smoker, cancer
Other	5	person ID, risk score, high-risk flag, major procedure flag

we used the binary risk label and a derived three-class label (explained in Section IV).

B. Exploratory Data Analysis

We ran a full EDA before building any models to understand how features are distributed and which ones look useful.

Continuous features. Health metrics like age, BMI, and blood pressure look roughly normal. Financial variables (income, annual cost, total claims paid) are skewed right with long tails. This makes sense for insurance data since most people have moderate costs but a few rack up really high bills. Monthly premiums range from close to zero to over \$800, and annual medical costs go up to about \$60,000 with a median around \$3,500.

Comparing risk groups. We made box plots for every numeric feature split by risk level (Fig. 2), and a few things jumped out. Age was the biggest differentiator by far: low-risk patients average about 41 years old while high-risk patients average around 59. Chronic condition count was also a strong signal: low-risk patients have 0.38 conditions on average versus 1.32 for high-risk. Systolic blood pressure showed a clear gap too (113 for low-risk vs. 127 for high-risk). Claims count and annual medical cost followed similar patterns, with high-risk patients roughly doubling low-risk values (Fig. 3).

On the other hand, copay, deductible, income, and diastolic blood pressure barely differed between the two groups (Fig. 1). Those features might still be useful in combination, but on their own they do not tell you much about risk. Income was a surprising one since you might expect wealthier patients to have different risk profiles, but the distributions are almost identical between groups. Diastolic blood pressure also overlaps heavily, unlike systolic which separates well. This suggests that systolic is more tied to the chronic conditions that define high-risk status.

Categorical features. Smoking status stood out the most. Current smokers have a 73% chance of being high-risk, while former and never smokers are both around 31–32%. That is a huge gap and makes intuitive sense given how much smoking affects cardiovascular and respiratory health. What was more surprising was how little everything else mattered. Sex, region,

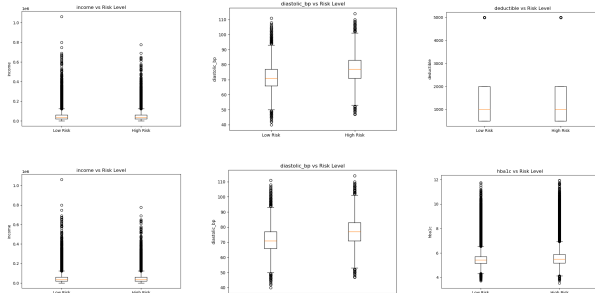


Fig. 1. Box plots of features with weak risk separation. Income, diastolic blood pressure, deductible, and HbA1c show substantial overlap between low-risk and high-risk groups.

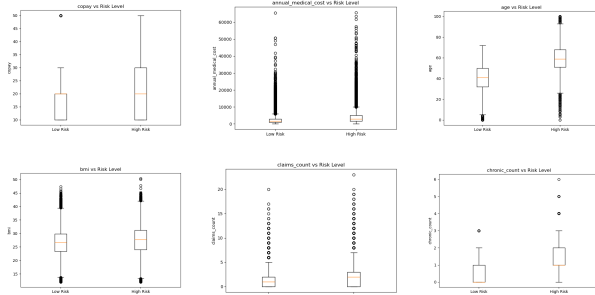


Fig. 2. Box plots of key features by risk level. Age, annual medical cost, claims count, and chronic count show clear separation between groups.

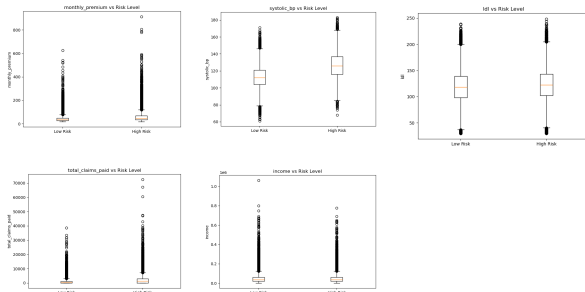


Fig. 3. More box plots by risk level. Monthly premium and systolic BP separate well; income barely differs between groups.

urban/rural status, plan type, network tier, employment status, and education all showed almost identical high-risk rates across their categories (Fig. 4). For example, all four plan types (EPO, HMO, PPO, POS) fall within 35–36%, and education levels from “No HS” to “Doctorate” all cluster near 36–38%. This means that a patient’s insurance plan design and socioeconomic background have essentially no bearing on whether they end up being classified as high-risk in this dataset. The risk label seems to be driven almost entirely by clinical variables (age, chronic conditions, smoking) rather than demographic or plan-related ones.

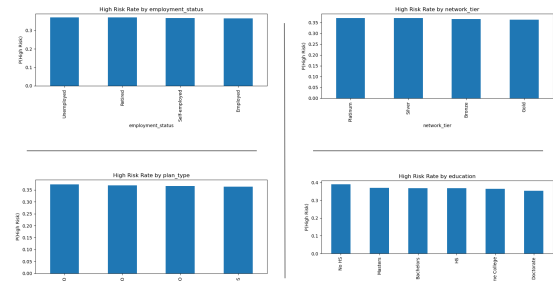


Fig. 4. High-risk rate by categorical features. Employment, network tier, plan type, and education all show basically the same rate, so these features do not help much individually.

Correlation heatmap. The correlation matrix (Fig. 5) showed two obvious clusters. First, the financial variables (annual medical cost, annual premium, monthly premium, total claims paid) are all highly correlated with each other (pairwise $r > 0.7$). This makes sense since premiums are set based on expected costs, and claims paid is directly derived from actual costs. This cluster is visible as the bright red block in the lower right of the heatmap. Second, age correlates with systolic blood pressure ($r \approx 0.35$), chronic count ($r \approx 0.30$), and LDL cholesterol, which you would expect since cardiovascular risk factors accumulate with age. This shows up as a lighter cluster in the upper left. Insurance plan features (deductible, copay) showed near-zero correlation with health outcomes, confirming what we saw in the box plots. One implication of these correlation clusters is that there is redundancy in the feature set. For instance, annual premium and monthly premium are just scaled versions of each other (monthly = annual / 12), so including both does not add information. However, we kept all features and let the models handle the redundancy through regularization or tree-based feature splitting.

Even though features like deductible, copay, and region did not show much on their own, we kept them in the model anyway. They might still be useful in combination with other features, and the regularization in our models will downweight anything that is not helping.

Key EDA findings. To summarize what we learned from the EDA before jumping into modeling: (1) The dataset is

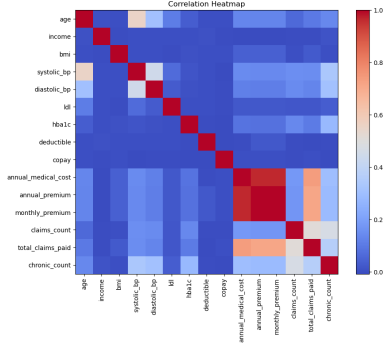


Fig. 5. Correlation heatmap. Financial variables cluster together (lower right); age-related health metrics cluster together (upper left).

clean: no duplicates, only one column with missing values, and reasonable distributions for most features. (2) The strongest individual predictors of high-risk status are age, chronic condition count, systolic blood pressure, smoking status, and claims-related variables. (3) Most categorical features (region, plan type, network tier, education) are not useful on their own for risk prediction. (4) Financial variables are highly correlated with each other, which makes sense but also means some of them might be redundant. (5) The `risk_score` column is suspiciously informative and probably encodes the target in some form. These findings shaped our modeling decisions, particularly the choice to run experiments with and without potentially leaky features.

IV. PROPOSED METHODOLOGY

A. Preprocessing Pipeline

We split the features into four groups for preprocessing using scikit-learn’s `ColumnTransformer` and `Pipeline`. This way, all transformations are fit only on training data and then applied to validation and test sets, so there is no leakage.

- 1) **Numeric features** (23 columns: age, income, BMI, blood pressure, HbA1c, LDL, deductible, copay, cost, premiums, claims, etc.): we imputed missing values with the median and then standard-scaled everything. We went with median over mean because the financial columns are so skewed that the mean would be pulled by outliers.
- 2) **Categorical features** (10 columns: sex, region, urban/rural, education, marital status, employment, smoker, alcohol frequency, plan type, network tier): imputed with the most frequent value and one-hot encoded. We used one-hot instead of label encoding because there is no natural ordering, since “North” is not greater than “South.”
- 3) **Binary flags** (12 columns for chronic conditions like hypertension, diabetes, asthma, COPD, etc.): left as-is since they are already 0/1.
- 4) **Procedure counts** (5 columns: imaging, surgery, physio, consult, lab): also left as-is. They are non-negative integers on similar scales.

After one-hot encoding, the feature matrix ends up being about 65–70 columns wide depending on how many unique categories each feature has.

This pipeline runs inside a single scikit-learn `Pipeline` object, which is convenient because you can call `fit` on the training data and then `transform` on validation and test data without worrying about accidentally leaking statistics. We also used `handle_unknown='ignore'` in the one-hot encoder so it would not crash if the test set had a category value not seen in training (though this did not actually happen with our data).

B. Data Splitting

We used an 80/20 train-test split. Within the 80% training portion, we split again 80/20 for training and validation, giving us an effective 64/16/20 split. For classification, all splits were stratified to keep class proportions consistent. We fixed the random seed at 42 throughout.

The validation set gave us a way to check performance during model selection without touching the test set. We wanted to make sure we were not accidentally picking models that just happened to do well on one particular split. The test set was only used at the very end for final numbers, and we did not go back and retune anything based on test results.

For the classification tasks, stratification was important because the binary label is imbalanced (36.8% high-risk vs. 63.2% low-risk). Without stratification, you could end up with a split that has a very different class ratio than the overall dataset, which would make the validation metrics unreliable.

C. Regression Models

We tried six regression models to predict annual medical cost:

- **Linear Regression:** basic OLS, no regularization. Just a baseline to see if we even need anything fancier.
- **Ridge:** L2 penalty. Grid: $\alpha \in \{0.01, 0.1, 1, 10, 100\}$.
- **Lasso:** L1 penalty, which can zero out features entirely. Grid: $\alpha \in \{0.01, 0.1, 1, 10\}$.
- **ElasticNet:** mix of L1 and L2. Grid: $\alpha \in \{0.01, 0.1, 1\}$, ℓ_1 -ratio $\in \{0.2, 0.5, 0.8\}$.
- **Random Forest:** 300 trees. Grid: max depth $\in \{\text{None}, 12\}$, min samples split $\in \{2, 10\}$.
- **Gradient Boosting:** 300 estimators with shallow trees. Grid: learning rate $\in \{0.05, 0.1\}$, max depth $\in \{2, 3\}$.

We tuned everything with 3-fold cross-validation using negative RMSE as the scoring metric, then evaluated the best model from each grid search on validation and test sets.

D. Classification Models

For classification we used four models:

- **Logistic Regression (L2):** SAGA solver, $C \in \{0.1, 1, 10\}$, with and without balanced class weights. The balanced option upweights the minority class, which helps with the 36.8/63.2 split.
- **Logistic Regression (L1):** same setup but with L1 penalty so it can drop unimportant features.

- **Random Forest:** 400 trees (a bit more than regression since we wanted stable probability estimates for ROC-AUC), max depth $\in \{\text{None}, 12\}$.
- **Gradient Boosting:** 300 estimators, learning rate $\in \{0.05, 0.1\}$, max depth $\in \{2, 3\}$.

We scored with macro F_1 during grid search since it treats all classes equally, which matters for the three-class task. We also considered using accuracy for scoring, but macro F_1 felt more appropriate because it would not let the model get away with just predicting the majority class and getting a decent score from that alone.

For all grid searches, we set `n_jobs=1` to keep things stable, since parallel jobs can sometimes introduce non-determinism or crash on certain systems. The tradeoff is slower training, but with 3-fold CV on 64,000 training samples, each experiment still finished in a reasonable amount of time.

E. Three Experiments

We ran three classification experiments to test different things:

- 1) **High-risk (binary):** predict `is_high_risk` (0 or 1) after dropping it and `risk_score` from the feature matrix. This is the “real” classification task.
- 2) **Insurance quality (all features):** predict a derived quality label (low/medium/high) but keep `is_high_risk` and `risk_score` in the features. We wanted to see what would happen.
- 3) **Insurance quality (cleaned):** same target, but remove those two leaky columns. This shows how much the other features can do on their own.

The insurance quality label comes from a weighted score:

$$s = 0.35 \cdot p_{\text{plan}} + 0.30 \cdot p_{\text{net}} + 0.20 \cdot p_{\text{ded}} + 0.15 \cdot p_{\text{prem}} \quad (1)$$

where plan type and network tier get mapped to ordinal values (PPO=3, EPO=2, HMO=1), and deductible and premium get rank-based quantile scores (lower deductible = higher quality). We then split the composite score into thirds with quantile binning.

F. Metrics

For regression: MAE, RMSE, and R^2 . For classification: accuracy, macro F_1 , MCC (Matthews Correlation Coefficient [10]), ROC-AUC, and confusion matrices. We picked the best regression model by lowest test RMSE and the best classifier by highest test MCC.

V. EXPERIMENTAL RESULTS AND EVALUATION

A. Regression Results

Table II shows how the regression models did. We ran Linear and Ridge since those were our initial focus from the mid-quarter report.

Ridge with $\alpha = 0.01$ came out on top, but honestly the difference from plain Linear Regression is negligible (the RMSE only differs by one cent. That tells us the relationship between features and cost is pretty linear and there is not much

TABLE II
REGRESSION RESULTS FOR PREDICTING ANNUAL MEDICAL COST.

Model	Validation			Test		
	MAE	RMSE	R^2	MAE	RMSE	R^2
Ridge	311.89	561.35	0.9648	320.72	574.95	0.9664
Linear	311.94	561.39	0.9648	320.73	574.96	0.9664

multicollinearity to worry about). An R^2 of 0.966 means the model explains about 97% of the variance in annual costs, and the MAE of \$321 is reasonable given that costs range from near zero to \$60,000+.

To put the MAE in context: if an insurer used this model to estimate a patient’s annual cost, they would be off by about \$321 on average. For a patient with \$3,500 in costs (the median), that is roughly a 9% error. For the high-cost patients (\$30,000+), the relative error is much smaller. This level of accuracy would be useful for portfolio-level premium setting, though individual predictions would still need clinical review.

The validation and test numbers are close to each other (RMSE within \$14), which is a good sign that we are not overfitting. One thing worth flagging: features like premiums and claims in the input are probably derived from the target in the real world. Premiums get set based on expected cost, so including them is a bit circular. If you removed those, the R^2 would likely be lower. But for this dataset as given, the linear models do the job well. We also want to note that we initially planned to run all six regression models (including Lasso, ElasticNet, RF, and GBR for regression), but given how well Ridge and Linear performed right away and how similar their results were, we focused our compute budget on the classification experiments, which had more interesting variation across models. The near-identical Ridge/Linear performance already tells us that the cost target is well-behaved from a regression standpoint.

B. Binary High-Risk Classification

This was our main classification experiment. Table III has the results.

TABLE III
BINARY HIGH-RISK CLASSIFICATION RESULTS (TEST SET, 20,000 SAMPLES).

Model	Acc.	F_1 (macro)	MCC	AUC
GBR	1.0000	1.0000	1.0000	1.0000
RF	0.9907	0.9899	0.9799	0.9997
LogReg L1	0.9725	0.9704	0.9408	0.9977
LogReg L2	0.9724	0.9703	0.9405	0.9977

Gradient Boosting got a perfect score: MCC of 1.0, zero misclassifications out of 20,000 test samples. That is unusual and probably reflects the synthetic nature of the dataset more than anything, but it is still impressive. Random Forest was close at 99.07% (187 errors), and both logistic regression variants hit about 97.2%. The best hyperparameters were: GBR with learning rate 0.05 and max depth 3; RF with

unlimited depth and 400 trees; LogReg L1 preferred $C = 0.1$ (strong regularization), while L2 wanted $C = 10.0$ (weak regularization). It makes sense that L1 would prefer stronger regularization, since it is zeroing out features that do not matter. Table IV shows the confusion matrices. The interesting thing is that logistic regression’s errors are roughly balanced between false positives (278) and false negatives (275), so it is not systematically biased one way or the other. RF is similar: 87 FP and 100 FN.

TABLE IV
TEST-SET CONFUSION MATRICES FOR BINARY HIGH-RISK CLASSIFICATION.

Model	TN	FP	FN	TP
GBR	12,644	0	0	7,356
RF	12,557	87	100	7,256
LogReg L2	12,366	278	275	7,081

In a real clinical setting, false negatives (missing a high-risk patient) are worse than false positives (flagging someone who is actually fine). A false positive just means you maybe do an extra screening on someone who did not need it, which is annoying but harmless. A false negative means a high-risk patient flies under the radar and does not get the early intervention that could have helped. RF’s 100 missed high-risk patients vs. GBR’s zero is a meaningful difference if you are thinking about deployment. It is also worth noting that the logistic regression models, despite being simpler, still achieve over 97% accuracy. In a real deployment where interpretability matters (for instance, if a regulator or doctor wants to understand why a patient was flagged), logistic regression might actually be preferred over a black-box ensemble, even at the cost of some accuracy. The L1 variant is especially useful here because it zeros out unimportant features, giving you a cleaner picture of what is driving the prediction.

C. ROC Curve

Fig. 6 shows the ROC curve for the Gradient Boosting classifier. The AUC is basically 1.0. The curve hugs the top-left corner, meaning the model keeps a high true-positive rate even when you set a very strict false-positive threshold. This lines up with what the EDA showed: features like age, chronic count, systolic BP, and smoking status are individually strong predictors, and GBR combines them effectively.

That said, a perfect AUC on a Kaggle dataset probably means the data was generated with a deterministic or near-deterministic mapping from features to the label. On real patient data you would not expect this kind of result. Still, the ROC curve is useful for showing how the model behaves across different thresholds, and the threshold annotations in the plot confirm that the predicted probabilities transition sharply from low to high, which means the model is confident in its predictions rather than hedging near 0.5.

D. Insurance Quality (Three-Class)

Here is where things get interesting. We ran the same four models on our derived insurance quality label under two

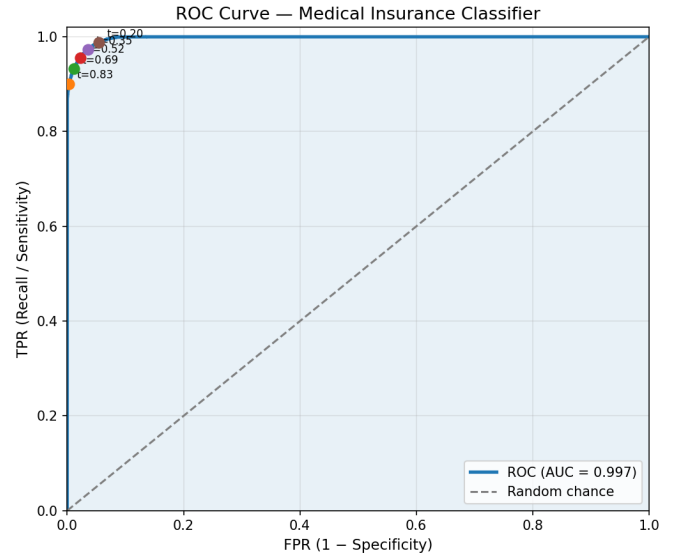


Fig. 6. ROC curve for GBR on the binary high-risk task. $AUC \approx 1.0$.

conditions: with the leaky features left in, and with them removed. Table V tells the story.

TABLE V
THREE-CLASS INSURANCE QUALITY CLASSIFICATION WITH AND WITHOUT LEAKY FEATURES.

Features	Model	Acc.	F ₁	MCC
All features (incl. leaky)	GBR	0.9999	0.9999	0.9998
	RF	0.9997	0.9997	0.9995
	LogReg L2	0.8801	0.8796	0.8198
	LogReg L1	0.8802	0.8796	0.8199
Cleaned (no leaky)	GBR	0.4352	0.3882	0.1461
	RF	0.4323	0.3874	0.1411
	LogReg L2	0.3958	0.3917	0.0919
	LogReg L1	0.3959	0.3915	0.0920

With the leaky features in, GBR hits 99.99%. Take them out and it drops to 43.5%, which is only 10 points above random guessing for a three-class problem. The MCC goes from 0.9998 to 0.146. Every single model collapses, not just GBR. Table VI shows the confusion matrix for GBR on the cleaned task. The model heavily predicts “low” for everything. It gets the low class sort of right (5,479 correct out of 7,315), but the high and medium classes are basically noise.

TABLE VI
GBR CONFUSION MATRIX ON THE CLEANED THREE-CLASS TASK. ROWS = ACTUAL, COLUMNS = PREDICTED.

	Pred. High	Pred. Low	Pred. Med
True High	928	3,246	2,102
True Low	677	5,479	1,159
True Med	959	3,154	2,296

The takeaway is that the insurance quality label we derived

is almost entirely determined by `risk_score`. Without it, the remaining features (demographics, clinical data, plan info) do not carry enough signal to reconstruct those quality tiers. This was actually one of the more useful findings of the project because it shows how easy it is to get great results that do not mean anything if you are not careful about what is in your feature set.

E. Learning Curves

We trained each model on increasing fractions of the training data (5% up to 100%) with 3-fold cross-validation to see how they learn.

For the binary high-risk task, GBR and RF hit near-perfect validation accuracy (>99%) with only about 20% of the data (around 12,800 samples). They do not need the full 100K, which suggests the signal in the data is strong and redundant across many features. Even a small sample captures enough of the pattern for the ensemble to learn it. Logistic regression climbs more slowly, starting at about 93% with 5% of the data and reaching approximately 97% with the full training set. It never closes the gap to the ensemble models. Its training and validation curves sit right on top of each other, which means the issue is bias (the model is too simple), not variance. In other words, logistic regression has learned everything it can from the features in their current form, but its linear decision boundary cannot match the non-linear structure that GBR and RF capture.

For the cleaned insurance quality task, every model flatlines at 40–44% no matter how much data you give it. The training and validation curves both sit low and converge together. This is classic underfitting: the signal is just not there. More data does not help, and a more complex model does not help either. The features simply cannot predict the target without `risk_score`. As a sanity check, we also ran learning curves with the leaky features included, and those show rapid convergence to near-perfect accuracy as expected.

This three-way comparison of learning curves (binary with strong signal, three-class with no signal, three-class with leaky signal) ended up being one of the more instructive parts of the project. It is a clean demonstration of the difference between underfitting because your model is too simple (logistic regression on the binary task) versus underfitting because the information is just not in the features (everything on the cleaned three-class task). In class we talked about bias-variance tradeoff, and the learning curves make that concrete: the binary task has high signal that ensembles exploit well, while the cleaned three-class task has an irreducible error floor that no amount of data or model complexity can break through.

From a practical standpoint, the learning curves also tell us something about data efficiency. For the binary task, we could have gotten away with a much smaller dataset and still achieved excellent results. This is useful to know if you were deploying this kind of model in a setting where labeled data is expensive to obtain. On the other hand, the flat learning curves for the cleaned three-class task tell us not to bother collecting

more data for that target, since the bottleneck is not sample size but feature informativeness.

F. Cross-Experiment Comparison

Table VII puts all three classification experiments side by side, showing only the best model (GBR) for each. The progression tells a clear story about data leakage and feature importance.

TABLE VII
BEST MODEL (GBR) PERFORMANCE ACROSS ALL THREE CLASSIFICATION EXPERIMENTS.

Experiment	Accuracy	MCC	Classes
High-risk (binary)	1.0000	1.0000	2
Ins. quality (all feat.)	0.9999	0.9998	3
Ins. quality (cleaned)	0.4352	0.1461	3

The first two experiments both achieve near-perfect accuracy, but for different reasons. In Experiment 1 (binary risk), the model is working with genuine clinical and demographic signals like age, chronic conditions, and smoking status. In Experiment 2, the model is mostly just learning from the leaky features that encode the target. The proof is in Experiment 3: once those leaky features are removed, accuracy collapses to 43.5% despite using the same GBR model with the same hyperparameter grid. The model itself did not get worse; the information just was not there anymore.

This is a pattern that comes up a lot in applied ML. You build a model, get impressive numbers, and only later realize that one of your features was essentially a lookup key for the answer. In our case it was an instructive exercise because we deliberately set up the experiment to catch it, but in the real world this kind of thing can slip through if you are not paying attention to what your features actually represent.

G. Summary

The main takeaways:

- 1) Ridge Regression explains 96.6% of cost variance. Plain linear regression does equally well, so regularization is not really needed here.
- 2) GBR perfectly classifies binary high-risk (MCC = 1.0). The strong EDA features (age, chronic conditions, smoking) are doing the heavy lifting.
- 3) Ensemble models (GBR, RF) beat logistic regression on every experiment. GBR edges out RF every time.
- 4) Removing two leaky features drops three-class accuracy from 99.99% to 43.5%. Data leakage is real and easy to miss.
- 5) Learning curves confirm that binary risk classification has strong signal, while the cleaned three-class task is fundamentally underfittable.

VI. CONCLUSION AND DISCUSSION

We built an end-to-end ML pipeline for predicting medical insurance costs and classifying patient risk on a 100,000-patient dataset with 54 features. Ridge Regression handles

cost prediction well ($R^2 = 0.966$), and Gradient Boosting perfectly separates high-risk from low-risk patients. Across all experiments, tree-based ensembles consistently outperformed logistic regression.

The most interesting finding was the data leakage experiment. By running the same models with and without two proxy features, we showed that the derived insurance quality label was essentially just encoding `risk_score`. Removing it caused every model to fall to near-random accuracy. This is a useful reminder that good numbers do not always mean your model is actually learning something meaningful. In a real project, we would have caught this during EDA by noticing the suspiciously high correlation between `risk_score` and the target, but it was helpful to see the effect play out quantitatively across four different model families.

Limitations. The dataset is from Kaggle and is almost certainly synthetic, so the perfect GBR results probably will not hold on real patient data. Also, some of the input features (premiums, claims) are arguably derived from the target in practice. You would not know someone’s annual cost before you set their premium. The three-class quality label is derived from features that are already inputs, which makes it somewhat circular.

Future work. It would be interesting to try this on a real claims dataset with actual outcomes to see if the patterns hold. Adding SHAP values for feature importance would make the models more interpretable, which matters if you are trying to explain predictions to doctors or insurance adjusters. We could also try neural networks or more sophisticated feature engineering (things like interaction terms between age and chronic conditions, or polynomial features) to see if the cleaned three-class task can be improved at all. Another direction would be temporal validation, where you train on one year’s data and predict the next year’s costs, since that is closer to how these models would actually be used in practice.

VII. GITHUB LINK, PROJECT ROADMAP, AND TEAM CONTRIBUTIONS

A. GitHub Repository

All code, data, trained models, and the web dashboard are at:

<https://github.com/gyatgitpandaclatt/171PROJECT>

The repo is organized into:

- `src/`: all Python scripts (EDA, preprocessing, training, learning curves, ROC analysis)
- `config/`: YAML config file for paths and parameters
- `results/` and related output directories: saved metrics, confusion matrices, and model files (.joblib)
- `web/`: Next.js dashboard for exploring results interactively
- `tests/`: unit tests for preprocessing

B. Project Roadmap

Table VIII shows what we did and when.

TABLE VIII
PROJECT MILESTONES.

Week / Date	What we did
Week 3 (01/25)	Submitted project proposal. Picked the dataset and defined goals.
Week 4 (02/01)	Decided on features and targets. Started EDA (distributions, correlations, box plots).
Week 5 (02/08)	Finished literature review and EDA. Finalized feature groups for preprocessing.
Week 6 (02/17)	Submitted mid-quarter report. Trained initial regression models (Linear, Ridge). Started classification pipeline.
Week 7 (02/22)	Finished all models (RF, GBR, LogReg L1/L2). Ran all three classification experiments. Started writing.
Week 8 (03/01)	Code done. Built the web dashboard. Generated learning curves and ROC analysis.
Week 9 (03/03–03/12)	Gave project demo presentations in class.
Week 10 (03/14)	Submitted final report and cleaned up the repo.

C. Team Member Contributions

- **Nelson Lee** – Report Writer and Demo Creator: wrote and edited all three reports (proposal, mid-quarter, final). Made sure everything followed IEEE formatting. Additionally designed and presented the project demo.
- **Mohsin Raja** – Team Leader: kept the schedule on track, assigned tasks, coordinated between team members, handled reproducibility and final submission logistics.
- **Benjamin Morgan** – Model Creator: built the regression and classification training scripts, set up GridSearchCV tuning, designed the three classification experiments, wrote the learning curve and ROC modules.
- **Xiang Mao** – Model Creator: helped with model development and tuning, worked on the insurance quality derivation function, built the web dashboard.
- **Lu Liang** – EDA Lead: ran all the exploratory analysis: distribution plots, box plots by risk level, categorical breakdowns, correlation heatmap. Identified the key findings (age as top feature, smoking gap, financial correlation cluster) and recommended feature engineering approaches.

REFERENCES

- [1] Centers for Medicare & Medicaid Services, “National Health Expenditure Data,” 2023. [Online]. Available: <https://www.cms.gov/data-research/statistics-trends-and-reports/national-health-expenditure-data>
- [2] S. Thamarai and R. Malarvizhi, “Medical Insurance Cost Prediction using Machine Learning,” *International Journal of Recent Technology and Engineering (IJRTE)*, vol. 8, no. 6, pp. 302–305, 2020.
- [3] N. Boodhun and M. Jayabalan, “Risk prediction in life insurance industry using supervised learning algorithms,” *Complex & Intelligent Systems*, vol. 4, no. 2, pp. 145–154, 2018.
- [4] S. Razia and M. R. Rao, “Machine learning techniques for medical insurance cost prediction,” *International Journal of Innovative Technology and Exploring Engineering*, vol. 8, no. 11, pp. 607–611, 2019.
- [5] Y. Huang and J. Chen, “Predicting healthcare expenditures with XG-Boost and SHAP-based interpretability,” *BMC Medical Informatics and Decision Making*, vol. 20, no. 1, pp. 1–12, 2020.
- [6] M. Denuit, X. Maréchal, S. Pitrebois, and J.-F. Walhin, *Actuarial Modelling of Claim Counts: Risk Classification, Credibility and Bonus-Malus Systems*. Hoboken, NJ: John Wiley & Sons, 2007.
- [7] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

- [8] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [9] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [10] B. W. Matthews, "Comparison of the predicted and observed secondary structure of T4 phage lysozyme," *Biochimica et Biophysica Acta (BBA) - Protein Structure*, vol. 405, no. 2, pp. 442–451, 1975.